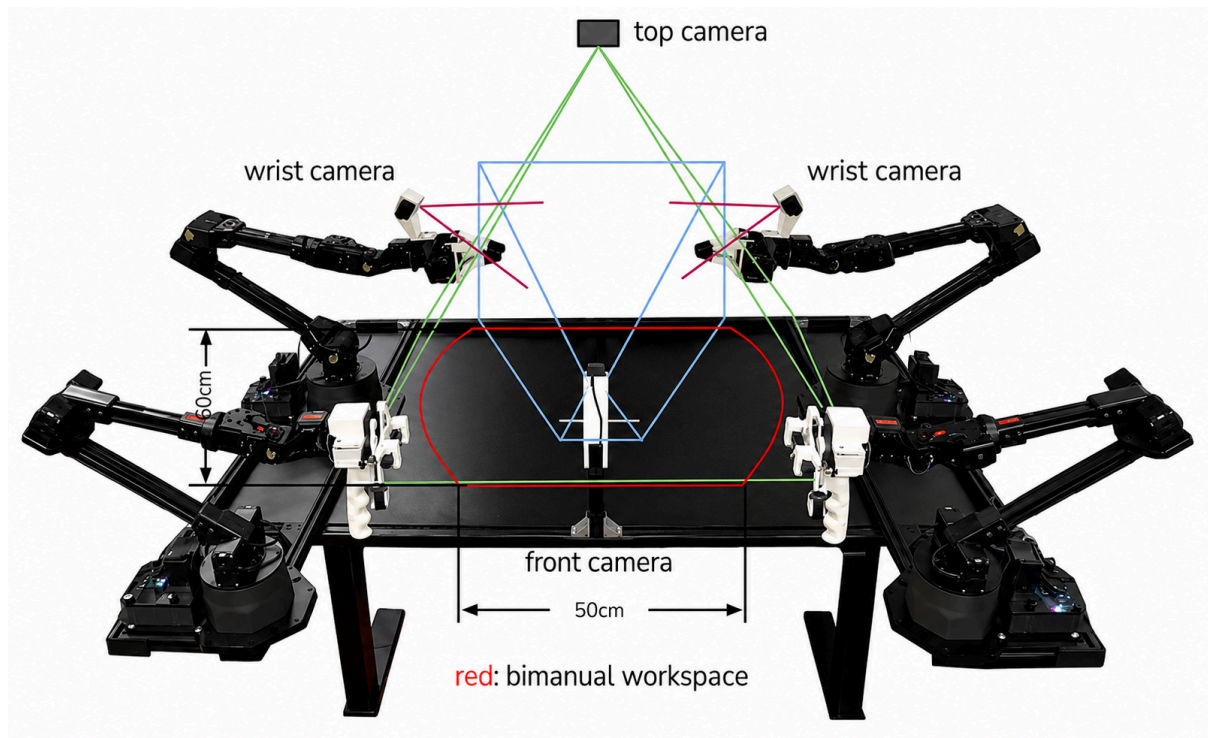


ALOHA Robot Overview

ALOHA (A Low-cost Open-source Hardware System for Bimanual Teleoperation) is a robotic platform developed for collecting demonstrations and learning complex two-handed manipulation tasks. The original system uses four robot arms and four cameras, connected to a control computer.

Check the official paper: <https://arxiv.org/abs/2304.13705>



1. Leader / Master Arms

There are two leader arms:

- Left Leader Arm
- Right Leader Arm

These are the arms directly manipulated by the operator during teleoperation. The operator moves them by hand, and their joint movements are transferred to the corresponding follower arms. The original ALOHA documentation refers to these as the master robots.

2. Follower / Puppet Arms

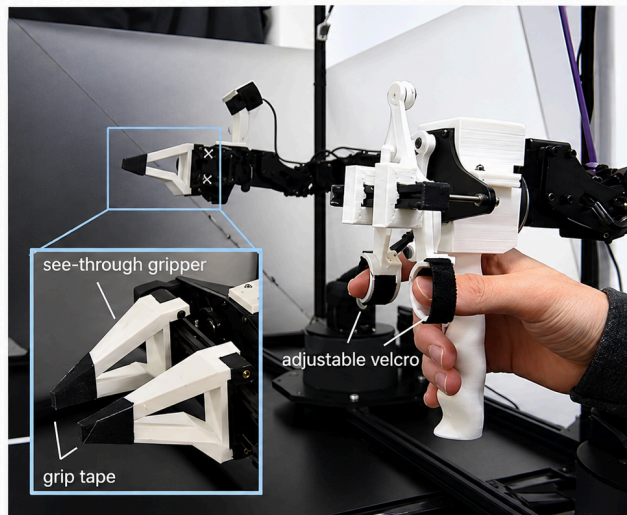
There are two follower arms:

- Left Follower Arm
- Right Follower Arm

These are the actual task-performing arms. They reproduce the motion commanded through the leader arms and interact with objects in the workspace. The original documentation calls these the puppet robots.

3. Grippers

Each arm includes a gripper. During teleoperation, the operator can control the follower grippers using the grippers on the leader arms. The grippers are used for manipulation tasks such as grasping, moving, inserting, and releasing objects.



ViperX 6dof Arm (follower)

#Dofs	6+gripper
Reach	750mm
Span	1500mm
Repeatability	1mm
Accuracy	5-8mm
Working Payload	750g

4. Cameras

The standard ALOHA configuration supports four camera views:

- **High Camera** — Provides an overview of the workspace.
- **Low Camera** — Observes the workspace from a lower viewpoint.
- **Left Wrist Camera** — Provides a close-up view from the left follower arm.
- **Right Wrist Camera** — Provides a close-up view from the right follower arm.

These camera observations can be recorded together with robot joint states and actions for imitation-learning datasets.

5. Control Computer

The control computer connects to all robot arms and cameras and runs the ALOHA software, ROS, teleoperation, data collection, and learned robot policies. The original ALOHA setup communicates with **four robots and four cameras simultaneously**.

ALOHA Operation Guide — GR00T

1. Start the ALOHA Robot

Tab 1 — Robot / ROS

Go to the ALOHA Docker directory:

```
cd ~/aloha/aist-aloha/resources/default  
docker start -ai aloha_pc
```

Note: If the arms are suddenly inactive, stop the container first:

```
docker stop aloha_pc
```

Then restart the environment.

If needed, run:

```
bash RUN_DOCKER_CONTAINER.sh
```

Note: This step can be skipped if the Docker container is already running correctly.

Set the RealSense camera serial numbers:

```
export RS_SERIAL_HIGH=109422063295  
export RS_SERIAL_LOW=109522061639
```

Source ROS and the Interbotix workspace:

```
source /opt/ros/noetic/setup.sh  
source ~/interbotix_ws/devel/setup.sh
```

Launch the ALOHA system:

```
roslaunch aist-aloha aist_teleop.launch
```

CAREFUL FOR ROBOT FALLING!!!

Sometimes one or two arms cannot be successfully powered on, as indicated by the status lights. Check the status lights and reconnect the cables if necessary, then rerun:

```
roslaunch aist-aloha aist_teleop.launch
```

2. Robot Testing

Tab 2 — Teleoperation

Open another Docker terminal:

```
bash RUN_DOCKER_CONTAINER.sh
```

Go to:

```
cd ~/interbotix_ws/src/aist-aloha/scripts
```

Test the left arm:

```
python3 one_side_teleop.py left
```

Test the right arm:

```
python3 one_side_teleop.py right
```

Move the robot to the sleep position:

```
python3 sleep.py
```

3. Data Collection

Go to:

```
cd ~/interbotix_ws/src/aist-aloha/scripts
```

Record GR00T demonstration data:

```
python3 record_episodes_gr00t.py \  
  --task_name put_soyjoy_on_the_shelf \  
  --episode_idx 0
```

Start from index 0 and increase the episode index for each new demonstration.

The collected data is stored inside the Docker environment at:

/root/data/put_soyjoy_on_the_shelf/

Perturbed Trajectory

If needed:

```
python3 execute_perturbed_trajectory.py \  
  --input_dir put_soyjoy_on_the_shelf
```

4. Check Collected Data

Tab 3 — Dataset

Open another Docker terminal:

```
bash RUN_DOCKER_CONTAINER.sh
```

Go to:

```
cd ~/data/
```

Use this directory to inspect the recorded demonstrations.

5. Download Data from Docker

Example:

```
docker cp \  
  aloha_pc:/root/data/put_object_on_the_shelf \  
  ./put_object_on_the_shelf
```

For SoyJoy:

```
docker cp \  
  aloha_pc:/root/data/put_soyjoy_on_the_shelf \  
  ./put_soyjoy_on_the_shelf
```

6. Upload Data to Docker

Upload a complete dataset:

```
docker cp \  
  ./put_soyjoy_on_the_shelf \  
  aloha_pc:/root/data/put_soyjoy_on_the_shelf
```

Upload a specific episode:

```
docker cp \
  ./Downloads/put_soyjoy_on_the_shelf/episode_5.hdf5 \
  aloha_pc:/root/data/put_soyjoy_on_the_shelf/
```

7. GR00T Model Training

After preparing the demonstration dataset, fine-tune the GR00T model using the converted GR00T dataset.

Example — Training on the “SoyJoy” Dataset

```
python scripts/gr00t_finetune.py \
  --dataset-path ./demo_data/put_soyjoy_on_the_shelf_gr00t/ \
  --num-gpus 1 \
  --output-dir ./demo_data/soyjoy-new-10k-lvar-checkpoints \
  --max-steps 10000 \
  --data-config aloha_qpos_4cam \
  --video-backend torchvision_av
```

The trained checkpoints will be saved to:

./demo_data/soyjoy-new-10k-lvar-checkpoints/

Tip: Use a larger `--max-steps` value for more complex datasets or longer trajectories.

8. GR00T Model Evaluation

After training, evaluate the model on the dataset before running it on the real robot.

Example — Evaluate the SoyJoy Model

```
python scripts/eval_policy.py \
  --plot \
  --embodiment_tag new_embodiment \
  --model_path ./demo_data/soyjoy-new-10k-lvar-checkpoints/ \
  --data_config aloha_qpos_4cam \
  --dataset_path
  ./ALOHATaskCompressedData/put_soyjoy_on_the_shelf_gr00t/ \
  --video_backend torchvision_av \
  --modality_keys qpos \
  --trajs 0
```

The `--plot` option visualizes the evaluation results, while `--trajs 0` selects trajectory 0 from the dataset for evaluation.

Use this step to check the trained policy before proceeding to real-robot testing.

9. Start the Robot Server

Server — Docker / ALOHA

Go to the Docker environment:

```
cd ~/aloha/aist-aloha/resources/default
```

Run:

```
bash RUN_DOCKER_CONTAINER.sh
```

Then go to:

```
cd ~/interbotix_ws/src/aist-aloha/scripts
```

Start the four-camera ZMQ robot server:

```
python3 robot_server_zmq_4cam.py
```

If a Python package is missing, install it using `pip` as required.

Check whether port 6000 is active:

```
ss -tulpn | grep 6000
```

If an old process is occupying the port, terminate it:

```
kill -9 <PID>
```

10. Run GR00T Policy

Local — Ubuntu / GR00T

Go to:

```
cd ~/Desktop/AIST/Muhammad/Isaac-GR00T/scripts
```

Activate the environment:

```
source ~/muhammad-ts/bin/activate
```

All-Tasks Model

```
python3 run_policy_zmq_4cam.py \  
  --host localhost \  
  --port 6000 \  
  --model_path ../demo_data/all-tasks-checkpoints/ \  
  --data_config aloha_qpos_4cam \  
  --embodiment_tag new_embodiment \  
  --use_cuda
```

SoyJoy Model

```
python3 run_policy_zmq_4cam.py \  
  --host localhost \  
  --port 6000 \  
  --model_path ../demo_data/soyjoy-new-10k-1var-checkpoints/ \  
  --data_config aloha_qpos_4cam \  
  --embodiment_tag new_embodiment \  
  --use_cuda
```

Instruction-Conditioned Model

```
python3 run_policy_zmq_4cam_inst.py \  
  --host localhost \  
  --port 6000 \  
  --model_path ../demo_data/object-new-14k-inst-checkpoints/ \  
  --data_config aloha_qpos_4cam_inst \  
  --embodiment_tag new_embodiment \  
  --use_cuda \  
  --instruction "Put Soyjoy on the shelf"
```

Groot Workflow

The GR00T workflow starts by launching the ALOHA robot and cameras inside Docker, checking that all four arms are properly powered, and testing teleoperation. Demonstration data is then collected and stored in `/root/data/`. After data collection, the ZMQ robot server is started inside Docker, while the GR00T policy is executed from the local Ubuntu environment.

ALOHA Operation Guide — ORPA

1. Start the ALOHA Robot

The following procedures are the same as the GR00T setup:

- Start the ALOHA Docker environment
- Start `aloha_pc`
- Restart the container if the arms suddenly become inactive
- Set the RealSense camera serial numbers
- Source ROS and the Interbotix workspace
- Launch `aist_teleop.launch`
- Check the arm status lights
- Reconnect cables if one or more arms fail to power on
- Test the left and right leader–follower arms
- Move the robot to the sleep position

Follow the corresponding sections in: **ALOHA Robot Operation Guide — GR00T**

2. ORPA Data Collection

Inside the ALOHA Docker environment:

```
cd ~/interbotix_ws/src/aist-aloha/scripts
```

Record demonstration data:

```
python3 record_episodes_orpa.py \  
  --task_name orpa_pick_and_place_chocolate \  
  --episode_idx 0
```

Start from episode index `0` and increase the index for every new demonstration.

The collected data is stored at:

`/root/data/orpa_pick_and_place_chocolate/`

3. Dataset Transfer

Dataset inspection and Docker file transfer are performed in the **same way as GR00T**.

For ORPA, download the dataset using:

```
docker cp \  
  \
```

```
aloha_pc:/root/data/orpa_pick_and_place_chocolate \  
./orpa_pick_and_place_chocolate
```

If necessary, upload it back using:

```
docker cp \  
./orpa_pick_and_place_chocolate \  
aloha_pc:/root/data/orpa_pick_and_place_chocolate
```

4. Train ACT Policy

After collecting and preparing the ORPA demonstration dataset, train the ACT policy using:

```
python3 imitate_episodes.py \  
--task_name orpa_pick_and_place_chocolate \  
--ckpt_dir ./checkpoints/orpa_pick_and_place_chocolate_discrete \  
--policy_class ACT \  
--kl_weight 10 \  
--chunk_size 100 \  
--batch_size 8 \  
--hidden_dim 256 \  
--dim_feedforward 3200 \  
--num_epochs 6000 \  
--lr 1e-5 \  
--seed 0 \  
--noise_data discrete
```

After training, select the appropriate checkpoint for real-robot rollout. The trained checkpoints will be saved to:

./checkpoints/orpa_pick_and_place_chocolate_discrete/

5. Train ORPA Feedback Modules

After training the base ACT policy, train the feedback modules using the ORPA dataset.

```
python3 train_feedback_modules_transformer.py \  
--task_name orpa_pick_and_place_chocolate \  
--ckpt_dir ./checkpoints/orpa_pick_and_place_chocolate_discrete \  
--policy_class ACT \  
--kl_weight 10 \  
--chunk_size 100 \  
--batch_size 8
```

```
--hidden_dim 256 \  
--dim_feedforward 3200 \  
--num_epochs 2000 \  
--lr 1e-5 \  
--fe_pu_lr 1e-5 \  
--seed 0 \  
--dataset_dir ./datasets/orpa_pick_and_place_chocolate_discrete \  
--feedback_vocab  
good,too_left,too_right,too_far,too_close,too_high,too_low \  
--realtime_plot \  
--noise_data discrete
```

The feedback module uses the following labels: `good`, `too_left`, `too_right`, `too_far`, `too_close`, `too_high`, `too_low`. This step trains the ORPA feedback modules on top of the base ACT policy so that feedback can be used for policy correction.

6. Start the Robot Server

The ZMQ robot server setup is the **same as GR00T**.

Inside the ALOHA Docker environment:

```
cd ~/interbotix_ws/src/aist-aloha/scripts  
python3 robot_server_zmq_4cam.py
```

The port checking and process termination procedures are also the same:

```
ss -tulpn | grep 6000  
kill -9 <PID>
```

7. Run ORPA / ACT Policy

This is the main difference from the GR00T workflow. On the local Ubuntu machine:

```
cd ~/Desktop/AIST/Muhammad/act
```

Activate the ACT environment:

```
conda activate aloha
```

Run the trained ORPA policy:

```
python3 run_policy_zmq_4cam.py \
  --host localhost \
  --port 6000 \
  --model_path
  ./checkpoints/orpa_pick_and_place_chocolate_discrete_4000/policy_best.ckpt \
  --stats_path
  ./checkpoints/orpa_pick_and_place_chocolate_discrete_4000/dataset_stats.pkl \
  --camera_names cam_high,cam_left_wrist,cam_low,cam_right_wrist \
  --use_cuda
```

There are two leader arms:

- cam_high
- cam_left_wrist
- cam_low
- cam_right_wrist

Make sure `robot_server_zmq_4cam.py` is already running before starting the ACT with ORPA policy.

ORPA Workflow

The robot startup, teleoperation testing, Docker setup, camera setup, dataset transfer, and ZMQ server procedures are the same as GR00T. The main ORPA-specific steps are:

Collect ORPA Demonstrations, Start Shared ZMQ Server, and Run ACT / ORPA Policy.